The Treacherous Transpiler: AI Assistance & Code-Comprehension Prompt Log

—

BreaQ 2026 — Challenge 2 Documentation of Conceptual & Repository Code Queries

Team: $\mu - Qbit$
Participants: Richard Watry, Dylane Mahazoasy
Date: October 25, 2026

October 25, 2026

Abstract

This document records the prompts used during Challenge 2 (*The Treacherous Transpiler*) to understand unfamiliar Qiskit transpiler internals, DAG manipulation methods, and Python idioms found in the provided repository (`breaq_ch2/service.py` and `02_treacherous_transpiler.ipynb`). All experimental design, fault localization, trigger characterization, minimal circuit synthesis, and verifier implementation were performed independently by the team.

1 Scope and Methodology of AI Usage

While inspecting the provided local compiler mock-up (`breaq_ch2/service.py`) and the challenge notebook scaffold (`02_treacherous_transpiler.ipynb`), we used an LLM strictly as a technical reference to clarify Qiskit API details and specific lines of code in the repository. In particular, our queries focused on:

1. Understanding how Qiskit's `TransformationPass` and `DAGCircuit` methods (`dag.nodes_on_wire`, `qargs`, and `dag.substitute_node`) work in `breaq_ch2/service.py`.
2. Understanding Python's sliding-window `zip(nodes, nodes[1:], nodes[2:])` idiom used in `_EchoBalance.run`.
3. Understanding how `PassManager` chains custom passes with `transpile(..., basis_gates=BASIS_GATES)` in `CompilerService.trace`.
4. Clarifying the difference between `Statevector.from_instruction`, `Operator.equiv`, and `circuit_digest` (QPY serialization).
5. Exploring the quantum-information background of Euler basis lowering ($\{\texttt{rz}, \texttt{sx}, \texttt{x}, \texttt{cx}\}$) and global vs. relative phase.

2 Repository Code Comprehension Prompts

2.1 Prompt 1 — `dag.nodes_on_wire` and `zip` in `service.py` (Lines 75–80)

Context in Repository: While reading `breaq_ch2/service.py` (which explicitly notes on Line 3 that reading the file is allowed), we encountered Qiskit's internal `DAGCircuit` traversal inside `_EchoBalance.run(self, dag)`.

Listing 1: Lines 69–80 of `breaq_ch2/service.py`.

```
class _EchoBalance(TransformationPass):
    """Vendor plugin used by the compiler build."""
```

```
    def __init__(self, seed: int):
        super().__init__()
        self.lower, self.upper = _profile(seed)

    def run(self, dag):
        for wire in dag.qubits:
            nodes = list(dag.nodes_on_wire(wire, only_ops=True))
            for left, middle, right in zip(nodes, nodes[1:], nodes[2:]):
                if (left.op.name, middle.op.name, right.op.name) != ('cx', 'rz', 'cx'):
                    continue
```

Prompt Used:

> *"In Qiskit's transpiler, I am reading a custom* `TransformationPass` *in* `breaq_ch2/service.py` *that runs* `nodes = list(dag.nodes_on_wire(wire, only_ops=True))` *and then loops* `for left, middle, right in zip(nodes, nodes[1:], nodes[2:]):`. *What does* `dag.nodes_on_wire(wire, only_ops=True)` *return compared to iterating over* `circuit.data`*, and how does* `zip(nodes, nodes[1:], nodes[2:])` *work in Python?"*

Takeaway Used by the Team: This explained two key points:

- Unlike `circuit.data`, which lists all instructions across the entire circuit in a single flat list, `dag.nodes_on_wire(wire, only_ops=True)` yields only the operation nodes (`DAGOpNode`) that touch that specific `Qubit` wire in topological order (excluding input/output boundary nodes because `only_ops=True`).
- In Python, `zip(nodes, nodes[1:], nodes[2:])` pairs elements $(i, i+1, i+2)$ to iterate over every contiguous 3-gate window along that wire.

2.2 Prompt 2 — `qargs` Indexing and `dag.substitute_node` in `service.py` (Lines 81–89)

Context in Repository: Immediately following the `zip` loop in `_EchoBalance.run`, the code checks `left.qargs` and calls `dag.substitute_node`.

Listing 2: Lines 81–89 of `breaq_ch2/service.py`.

```
                if tuple(left.qargs) != tuple(right.qargs):
                    continue
                if left.qargs[1] != wire or middle.qargs != (wire,):
                    continue
                angle = float(middle.op.params[0])
                if self.lower <= angle <= self.upper:
                    dag.substitute_node(middle, RZGate(-angle), inplace=True)
                    return dag  # first matching target-wire pattern only
        return dag
```

Prompt Used:

> *"Could you explain lines 81–88 of* `service.py`*? Specifically, on a* `DAGOpNode` *for a* `cx` *gate, what is stored in* `left.qargs`*, why does* `left.qargs[1] != wire` *check for the target qubit rather than the control qubit, and what does* `dag.substitute_node(middle, RZGate(-angle), inplace=True)` *do to the DAG?"*

Takeaway Used by the Team: In Qiskit, a two-qubit `cx(control, target)` node stores its quantum arguments as a 2-tuple `qargs = (control_qubit, target_qubit)`. Therefore, `left.qargs[0]` is the control wire and `left.qargs[1]` is the target wire. Checking `left.qargs[1] != wire` ensures that the current `wire` being inspected is the target of both `cx` gates, while `middle.qargs != (wire,)` ensures the middle operation is a 1-qubit gate on that same wire. Finally, `dag.substitute_node` replaces the `DAGOpNode` in place with a new `RZGate(-angle)` while preserving the wire connections.

2.3 Prompt 3 — `PassManager` and `transpile` Parameters in `service.py` (Lines 121–138)

Context in Repository: In `CompilerService.trace`, the service runs a list of custom passes followed by Qiskit's `transpile` function before recording snapshots.

Listing 3: Lines 127–138 of `breaq_ch2/service.py`.

```
        passes = [('01:normalization', RemoveBarriers()),
                  ('02:phase-convention', _PhaseConvention())]
        if build == 'suspect':
            passes.append(('03:vendor-echo-balance', _EchoBalance(self.seed)))
        for stage, transformation in passes:
            current = PassManager([transformation]).run(current)
            snapshots.append(_record(stage, current))
        current = transpile(current, basis_gates=BASIS_GATES, optimization_level=1,
                            approximation_degree=1.0, seed_transpiler=self.seed)
        validate_circuit(current)
        snapshots.append(_record('04:basis-lowering', current))
```

Prompt Used:

> *"In `CompilerService.trace` (lines 127–138 of `service.py`), how does `PassManager([transformation]).run(curr` convert between a `QuantumCircuit` and a `DAGCircuit` automatically, and what does `approximation_degree=1.0` mean inside the `transpile(...)` call at stage 04?"*

Takeaway Used by the Team: Qiskit's `PassManager.run(circuit)` automatically converts the input `QuantumCircuit` into a `DAGCircuit`, executes each `TransformationPass.run(dag)`, and converts the resulting DAG back into a `QuantumCircuit`. In `transpile`, setting `approximation_degree=1.0` instructs Qiskit to perform exact synthesis (no intentional unitary approximation errors during basis lowering to `['rz', 'sx', 'x', 'cx']`).

2.4 Prompt 4 — `circuit_digest` (QPY) vs. `exact_probabilities` in `service.py` (Lines 176–208)

Context in Repository: At the bottom of `breaq_ch2/service.py`, several helper functions are provided for comparing circuits, including `exact_probabilities`, `total_variation`, and `circuit_digest`.

Listing 4: Lines 176–180 and 204–208 of `breaq_ch2/service.py`.

```
def exact_probabilities(circuit: QuantumCircuit) -> dict[str, float]:
    """Exact zero-input, computational-basis probabilities, NOT full equivalence."""
    validate_circuit(circuit)
    return {str(k): float(v) for k, v in Statevector.from_instruction(circuit).
    probabilities_dict().items()}

def circuit_digest(circuit: QuantumCircuit) -> str:
    """Byte-level QPY artifact identifier, not a semantic-equivalence test."""
    stream = io.BytesIO()
    qpy.dump(circuit, stream)
    return hashlib.sha256(stream.getvalue()).hexdigest()
```

Prompt Used:

> *"In `breaq_ch2/service.py`, `circuit_digest` uses `qpy.dump(circuit, stream)` and hashes the bytes with SHA-256, while `exact_probabilities` calls `Statevector.from_instruction(circuit).probabilit` What is stored in Qiskit's QPY binary format, and why does `Statevector.from_instruction` only depend on the $|0\ldots0\rangle$ input state?"*

Takeaway Used by the Team: QPY (`qiskit.qpy`) is Qiskit's binary serialization format for `QuantumCircuit` objects: it serializes the exact instruction list, register names, parameters, and metadata, so any gate decomposition changes the SHA-256 digest even when the unitary is preserved. Meanwhile, `Statevector.from_instruction(circuit)` initializes the n-qubit state to $|0^{\otimes n}\rangle$ and evolves it through the circuit, meaning it only computes the first column of the unitary matrix $U|0^{\otimes n}\rangle$.

3 Conceptual & Quantum-Information Curiosity Prompts

3.1 Prompt 5 — Global Phase vs. Relative Phase in `Operator.equiv`

Prompt Used:

> *"In Qiskit* `quantum_info`*, why does adding* `dag.global_phase + math.pi / 7` *(like in* `_PhaseConvention` *in* `service.py`*) still return* `True` *when checking* `Operator(c1).equiv(Operator(c2))`*, whereas changing the sign of an* `rz(theta)` *gate between two* `cx` *gates changes the unitary equivalence result?"*

Takeaway Used by the Team: Multiplying an entire standalone circuit by $e^{i\phi}$ scales every entry of the unitary matrix U by the same unit-modulus scalar $e^{i\phi}$, which produces physically indistinguishable measurement statistics across all input states and measurement bases; `Operator.equiv` checks whether $U_1^\dagger U_2 = e^{i\phi} I$. By contrast, $\mathrm{CX}_{0,1} R_z(\theta)_1 \mathrm{CX}_{0,1} = \exp(-i\frac{\theta}{2} Z_0 \otimes Z_1)$ applies opposite phases $e^{\mp i\theta/2}$ to the even-parity ($|00\rangle, |11\rangle$) and odd-parity ($|01\rangle, |10\rangle$) subspaces, which is a relative phase between orthogonal subspaces and therefore alters the unitary operator when $\theta \mapsto -\theta$.